



Counting Unique Even and Odd Numbers

Question 1: 2025 Jan OPPE 1 - Set 1 – Section 2

The Problem Statement

The Challenge

Write a function that takes a list of integers and counts unique even and odd numbers separately.

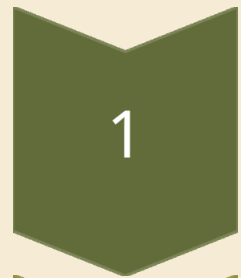
Your function should return a dictionary with counts for each category.

```
def count_unique_even_odd(l):  
    # Your code here  
    return {  
        "even": count_of_unique_even,  
        "odd": count_of_unique_odd  
    }
```



Note: This is a function-only question. No input statements or print statements are needed.

Understanding "Unique" Numbers



What We Start With

Original list containing duplicates: [1, 2, 2, 3, 4, 5, 5, 6, 9]



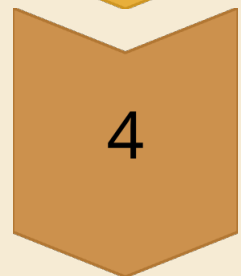
Even Numbers

Duplicates: 2 appears twice, so we count each number only once



Unique Even

Result: {2, 4, 6} $\Rightarrow$ Count = 3



Unique Odd

Result: {1, 3, 5, 9} $\Rightarrow$ Count = 4

Core Concepts Involved

This question tests your understanding of three fundamental Python concepts:



Even/Odd Check

Using the modulo operator `% 2` to determine if a number is divisible by 2



Removing Duplicates

Converting to a set to automatically eliminate duplicate values



Dictionary Creation

Building a dictionary to store and return results in the required format

Break it down step by step. Don't try to solve everything at once.

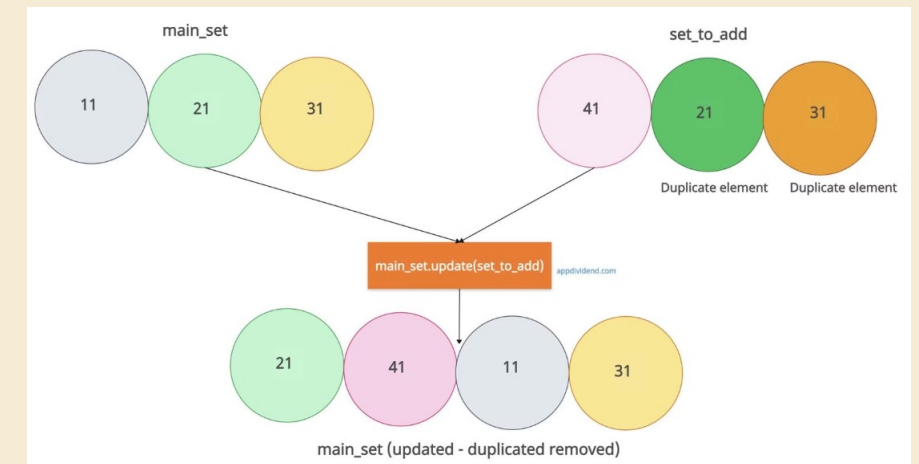
Step 1: Remove Duplicates

The Set Solution

Python provides a built-in tool called `set()` that automatically removes duplicates from a list.

```
l = [1, 2, 2, 3, 4, 5, 5, 6, 9]
unique_nums = set(l)
print(unique_nums)
# Output: {1, 2, 3, 4, 5, 6, 9}
```

Now we only work with unique numbers, making our counting logic much simpler.



Step 2: Separate Even and Odd

The Modulo Operator

After removing duplicates, we loop through each unique number and check if it's even or odd using `num % 2`.

Even Numbers

```
if num % 2 == 0:  
    # This number is even  
    even_count += 1
```

Odd Numbers

```
else:  
    # This number is odd  
    odd_count += 1
```

Building the Logical Structure



Convert to Set

Remove all duplicates from the input list



Initialize Counters

Set `even_count` and `odd_count` to zero



Loop Through Numbers

Iterate over each unique number in the set



Check Even/Odd

Use modulo operator to determine number type



Increment Counters

Update the appropriate counter based on the check



Return Dictionary

Create and return the result dictionary

The Clean Solution

Well-Structured Approach

- Clear variable names
- Single loop
- Proper indentation
- Expected return format

```
def count_unique_even_odd(l: list) -> dict:
    unique_nums = set(l)

    even_count = 0
    odd_count = 0

    for num in unique_nums:
        if num % 2 == 0:
            even_count += 1
        else:
            odd_count += 1

    return {
        "even": even_count,
        "odd": odd_count
    }
```


A More Pythonic Version

Generator Expressions Solution

For those comfortable with advanced Python features, you can use generator expressions with `sum()`:

```
def count_unique_even_odd(l: list) -> dict:
    unique_nums = set(l)

    even_count = sum(1 for num in unique_nums if num % 2 == 0)
    odd_count = sum(1 for num in unique_nums if num % 2 != 0)

    return {"even": even_count, "odd": odd_count}
```

📌 This is shorter and more advanced, but the first version is safer for beginners.

Common Mistakes & Best Practices

Common Mistakes

Forgetting Uniqueness

Counting duplicates by working with the original list instead of a set

Wrong Dictionary Keys

Using "Even" or "EVEN" instead of "even" in lowercase

Separate Loops

Creating two loops instead of one efficient iteration

Best Practices

- **Remove duplicates first**
Reduces logic complexity and errors
- **Use descriptive variable names**
`even_count` is better than `e` or `ec`
- **Return exactly what's asked**
Match dictionary keys exactly:
`"even"` and `"odd"`